
GUIDE 02

Docker Locally & Push to Azure

Build containers locally and deploy to Azure Container Apps, ACI, and AKS

Agentic AI Builder Expert Bootcamp

Batch 4.0 | April 2026

Table of Contents

1. Docker Fundamentals Refresher
2. Local Docker Setup
3. Writing Production Dockerfiles
4. Building and Testing Locally
5. Azure Container Registry (ACR) Setup
6. Pushing Images to ACR
7. Deploying to Azure Container Apps
8. Deploying to Azure Container Instances (ACI)
9. Deploying to Azure Kubernetes Service (AKS)
10. CI/CD with GitHub Actions
11. Monitoring and Troubleshooting

1. Docker Fundamentals Refresher

Docker packages applications into lightweight, portable containers that include everything needed to run: code, runtime, libraries, and system tools.

Key Concepts

- **Image:** A read-only template containing your app and its dependencies. Built from a Dockerfile.
- **Container:** A running instance of an image. Isolated, lightweight, and ephemeral.
- **Registry:** A storage and distribution service for Docker images (e.g., Docker Hub, Azure Container Registry).
- **Dockerfile:** A script of instructions to build a Docker image layer by layer.
- **Docker Compose:** A tool for defining multi-container applications using a YAML file.

2. Local Docker Setup

Install Docker Desktop

Download Docker Desktop from docker.com for Windows or macOS. For Linux, install Docker Engine directly.

```
# Verify installation
docker --version
docker compose version

# Test with hello-world
docker run hello-world
```

Install Azure CLI

```
# macOS
brew install azure-cli

# Windows (PowerShell)
winget install Microsoft.AzureCLI

# Linux
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

# Login
az login
```

3. Writing Production Dockerfiles

3.1 Python FastAPI Agent (Multi-stage Build)

```
# Dockerfile
# Stage 1: Build dependencies
```

```

FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2: Production image
FROM python:3.11-slim AS production
WORKDIR /app

# Copy installed packages from builder
COPY --from=builder /install /usr/local

# Copy application code
COPY . .

# Create non-root user for security
RUN useradd --create-home appuser
USER appuser

# Azure Container Apps uses PORT env var
ENV PORT=8000
EXPOSE $PORT

CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Tip: Multi-stage builds reduce final image size by 40-60%. The builder stage installs dependencies, and only the compiled packages are copied to the production stage.

3.2 Optimized .dockerignore

```

# .dockerignore
.git
.gitignore
__pycache__
*.pyc
.env
.venv
venv/
*.md
.pytest_cache
.mypy_cache
tests/
docs/
*.egg-info
.DS_Store

```

3.3 Docker Compose for Local Development

```

# docker-compose.yml
version: "3.9"
services:
  agent:
    build: .
    ports:
      - "8000:8000"
    env_file:

```

```
- .env
volumes:
- ./agent:/app/agent # Hot reload
depends_on:
- redis
redis:
image: redis:7-alpine
ports:
- "6379:6379"
postgres:
image: postgres:16-alpine
environment:
POSTGRES_DB: agentdb
POSTGRES_USER: agent
POSTGRES_PASSWORD: secret
ports:
- "5432:5432"
volumes:
- pgdata:/var/lib/postgresql/data
volumes:
pgdata:
```

4. Building and Testing Locally

Step 1: Build the Docker Image

```
docker build -t langgraph-agent:latest .

# With build arguments
docker build -t langgraph-agent:latest \
--build-arg PYTHON_VERSION=3.11 .
```

Step 2: Run the Container

```
docker run -d \
--name agent-local \
-p 8000:8000 \
-e OPENAI_API_KEY=sk-xxx \
langgraph-agent:latest
```

Step 3: Test the API

```
curl http://localhost:8000/health

curl -X POST http://localhost:8000/chat \
-H "Content-Type: application/json" \
-d '{"message": "Hello agent!"}'
```

Step 4: Debug if Needed

```
# View logs
docker logs agent-local -f

# Shell into container
```

```
docker exec -it agent-local /bin/bash

# Inspect container
docker inspect agent-local
```

5. Azure Container Registry (ACR) Setup

Step 1: Create a Resource Group

```
az group create \
  --name rg-agentic-ai \
  --location southeastasia
```

Step 2: Create Azure Container Registry

```
az acr create \
  --resource-group rg-agentic-ai \
  --name agenticairegistry \
  --sku Basic \
  --admin-enabled true
```

Step 3: Get Login Credentials

```
az acr credential show \
  --name agenticairegistry

# Login to ACR
az acr login --name agenticairegistry
```

6. Pushing Images to ACR

Step 1: Tag the Image for ACR

```
docker tag langgraph-agent:latest \
  agenticairegistry.azurecr.io/langgraph-agent:v1.0

docker tag langgraph-agent:latest \
  agenticairegistry.azurecr.io/langgraph-agent:latest
```

Step 2: Push to ACR

```
docker push agenticairegistry.azurecr.io/langgraph-agent:v1.0
docker push agenticairegistry.azurecr.io/langgraph-agent:latest
```

Step 3: Verify in ACR

```
az acr repository list --name agenticairegistry
az acr repository show-tags --name agenticairegistry \
  --repository langgraph-agent
```

Tip: Always push both a version-tagged image (v1.0) and a :latest tag. Use version tags for production rollbacks.

7. Deploying to Azure Container Apps

Azure Container Apps is the recommended option for serverless, auto-scaling agent deployments. It handles scaling to zero when idle.

Step 1: Create Container Apps Environment

```
az containerapp env create \  
  --name agent-env \  
  --resource-group rg-agentic-ai \  
  --location southeastasia
```

Step 2: Deploy the Container App

```
az containerapp create \  
  --name langgraph-agent \  
  --resource-group rg-agentic-ai \  
  --environment agent-env \  
  --image agenticairegistry.azurecr.io/langgraph-agent:v1.0 \  
  --registry-server agenticairegistry.azurecr.io \  
  --registry-username agenticairegistry \  
  --registry-password \  
  --target-port 8000 \  
  --ingress external \  
  --min-replicas 0 \  
  --max-replicas 5 \  
  --cpu 1.0 \  
  --memory 2.0Gi \  
  --env-vars OPENAI_API_KEY=secretref:openai-key
```

Step 3: Get the App URL

```
az containerapp show \  
  --name langgraph-agent \  
  --resource-group rg-agentic-ai \  
  --query properties.configuration.ingress.fqdn -o tsv
```

8. Deploying to Azure Container Instances (ACI)

ACI is the simplest option for quick, single-container deployments without orchestration.

```
az container create \  
  --resource-group rg-agentic-ai \  
  --name langgraph-agent-aci \  
  --image agenticairegistry.azurecr.io/langgraph-agent:v1.0 \  
  --registry-login-server agenticairegistry.azurecr.io \  
  --registry-username agenticairegistry \  
  --registry-password \  
  --cpu 2 --memory 4 \  
  --ports 8000 \  
  --ip-address public
```

```
--environment-variables \  
  OPENAI_API_KEY=sk-xxx \  
--restart-policy OnFailure
```

9. Deploying to Azure Kubernetes Service (AKS)

AKS provides full orchestration for complex multi-agent architectures needing custom networking, service meshes, and advanced scaling.

Step 1: Create AKS Cluster

```
az aks create \  
  --resource-group rg-agentic-ai \  
  --name aks-agentic \  
  --node-count 2 \  
  --node-vm-size Standard_B2s \  
  --attach-acr agenticaregistry \  
  --generate-ssh-keys
```

Step 2: Get Credentials

```
az aks get-credentials --resource-group rg-agentic-ai --name aks-agentic
```

Step 3: Create Kubernetes Deployment YAML

```
# k8s/deployment.yaml  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: langgraph-agent  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: langgraph-agent  
  template:  
    metadata:  
      labels:  
        app: langgraph-agent  
    spec:  
      containers:  
        - name: agent  
          image: agenticaregistry.azurecr.io/langgraph-agent:v1.0  
          ports:  
            - containerPort: 8000  
          env:  
            - name: OPENAI_API_KEY  
              valueFrom:  
                secretKeyRef:  
                  name: agent-secrets  
                  key: openai-api-key
```



```

resources:
  requests:
    cpu: "500m"
    memory: "1Gi"
  limits:
    cpu: "1000m"
    memory: "2Gi"
---
apiVersion: v1
kind: Service
metadata:
  name: langgraph-agent-svc
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 8000
  selector:
    app: langgraph-agent

```

Step 4: Apply and Verify

```

kubectl apply -f k8s/deployment.yaml
kubectl get pods
kubectl get svc langgraph-agent-svc

```

Azure Deployment Options Comparison

Feature	Container Apps	ACI	AKS
Complexity	Low	Very Low	High
Auto-scaling	Yes (0 to N)	No	Yes (HPA)
Scale to zero	Yes	No	No
Cost model	Per-second	Per-second	Per-node
Best for	Most agent deployments	Dev/test, simple APIs	Multi-agent systems
Networking	Built-in ingress	Public IP	Full control
Setup time	~10 minutes	~5 minutes	~30 minutes

10. CI/CD with GitHub Actions

```

# .github/workflows/deploy.yml
name: Build and Deploy to Azure
on:

```

```

push:
  branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to ACR
        uses: azure/docker-login@v1
        with:
          login-server: agenticairegistry.azurecr.io
          username: ${ secrets.ACR_USERNAME }
          password: ${ secrets.ACR_PASSWORD }

      - name: Build and Push
        run: |
          docker build -t agenticairegistry.azurecr.io/langgraph-agent:${ github.sha } .
          docker push agenticairegistry.azurecr.io/langgraph-agent:${ github.sha }

      - name: Deploy to Container Apps
        uses: azure/container-apps-deploy-action@v1
        with:
          resourceGroup: rg-agentic-ai
          containerAppName: langgraph-agent
          imageToDeploy: agenticairegistry.azurecr.io/langgraph-agent:${ github.sha }

```

11. Monitoring and Troubleshooting

Common Issues

Issue	Diagnosis	Fix
Container won't start	Check logs: az containerapp logs show	Verify image tag and env vars
ACR auth error	401 Unauthorized	Re-run az acr login or check service principal
Out of memory	OOMKilled in pod events	Increase memory limits or optimize agent state
Slow cold starts	Scale-to-zero warmup	Set min-replicas to 1 for critical services
DNS not resolving	Custom domain misconfigured	Verify CNAME records and TLS certificates